

Assignment 4

Question 1: You are working on a college project with 3 teammates. Everyone is changing code at the same time. Sometimes code gets overwritten. What tool/process will you use to manage this properly and why?

Answer:

To solve this issue, we should use a Distributed Version Control System (DVCS) like Git, hosted on a cloud service like GitHub or GitLab.

Each developer works in their own dedicated branch (e.g., feature/login, feature/dashboard) instead of pushing directly to the primary branch.

Before merging code into main, developers submit a PR to request review from teammates.

Why this approach:

- Prevents Overwriting: Changes are isolated in separate branches until reviewed.
- Conflict Management: Git automatically highlights conflicting edits, requiring developer resolution before merging.
- Traceability & Backups: Every change is logged with the developer's name, timestamp, and message, making it easy to revert to previous working states.

Question 2: You wrote some code and used git add but forgot to commit. Your system crashed. After restart, what will happen to your changes and why?

Answer:

My staged changes will remain safe in the .git database directory.

Why:

When we run git add, Git takes a snapshot of the files and writes them in the staging area. Since these changes are physically written to disk prior to committing, a system crash will not delete them unless the hard drive itself gets corrupted.

How to recover them after restart:

- Running git status will show the files still listed under "Changes to be committed".
- We can simply run git commit -m "Your commit message" to complete the commit.

Question 3: A developer says “I directly pushed code to main branch without testing and it broke production.” What Git workflow mistake did they make and how should it be improved?

Answer:

Mistakes Made:

- Direct Push to main: Pushing untested code directly to the primary branch without code review.
- Lack of Branch Protection: The repository did not restrict direct pushes to the production branch.

Improvements to Implement:

- Branch Protection Rules: Configure repository settings to block direct commits/pushes to main.
- Require Pull Requests (PRs): Force all changes to go through a PR with mandatory peer approvals.
- Environment Staging: Test code in a development or staging environment prior to deploying to production.

Question 4: While merging a feature branch into main, Git shows a conflict in the same file edited by two people. What steps will you take to resolve it?

Answer:

Steps to Resolve a Git Merge Conflict:

- Identify Conflict Files: Run `git status` to locate files listed under "Unmerged paths".
- Open Conflict File: Open the flagged file in a code editor (e.g., VS Code) and locate Git conflict markers:

```
<<<<<<< HEAD (Current Change)
Code      from      main      branch
=====
Code      from      feature  branch
>>>>>>> feature-branch (Incoming Change)
```

- Manual Resolution: Communicate with the co-developer if necessary, select the correct code logic, and remove all conflict markers.
- Stage Resolved Files: Mark the conflict as resolved by staging the file using `git add <file-name>`.
- Finalize Merge Commit: Complete the merge process by running `git commit -m "Resolved merge conflict in <file-name>".`
- Push Changes: Push the resolved merge commit back to the remote repository.

Question 5: You cloned a repository but you are unable to push changes to GitHub. What could be the possible reasons and how will you fix it?

Answer:

Reason	Solution
1. Lack of Write Permissions	You cloned a repo you don't own/have write access to. Fix: Request collaborator access, or fork the repo and push to your fork.
2. Authentication Failure	Invalid Personal Access Token (PAT) or SSH key. Fix: Generate a new PAT on GitHub or re-add SSH keys (ssh-add ~/.ssh/id_rsa).
3. Remote Branch is Ahead	New commits exist on GitHub that aren't on your local machine. Fix: Run <code>git pull origin main</code> to pull latest changes before pushing.
4. Pushing to Protected Branch	Main branch forbids direct pushes. Fix: Push to a feature branch (<code>git push origin feature-branch</code>) and open a Pull Request.

Question 6: You are asked to build a personal portfolio and host it online for free using GitHub. What steps will you follow from local setup to deployment?

Answer:

Step 1: Local Setup

- Create a local folder and add project files (index.html, style.css, script.js).
- Initialize Git locally and commit changes:

```
git init
git add .
git commit -m "Initial portfolio commit"
```

Step 2: Connect to GitHub

- Create a public repository on GitHub (e.g., portfolio or username.github.io).
- Link local repository to GitHub and push code:

```
git remote add origin <your-repo-url>
git push -u origin main
```

Step 3: Deployment via GitHub Pages

- Go to repository Settings on GitHub.
- Scroll to the Pages section under 'Code and automation'.

- Under Source, select Deploy from a branch.
- Set branch to main and folder to / (root), then click Save.
Wait a few minutes; GitHub will generate a live web URL.

Question 7: What is version control and why is it important in software development?

Answer:

Definition:

Version Control System (VCS) is a software tool that tracks, manages, and logs changes made to files/source code over time.

Importance in Software Development:

- Complete Revision History: Enables developers to view historical file changes, author info, and precise code modifications.
- Easy Rollbacks: If a bug is introduced, you can instantly revert code to a stable prior state.
- Safe Concurrent Experimentation: Branching lets developers build features without affecting working production code.
- Collaboration Enablement: Allows multiple developers to work simultaneously on the same codebase without overwriting work.

Question 8: Explain the difference between Git and GitHub.

Answer:

Feature	Git	GitHub
Type	Command-line Version Control Tool (Software).	Cloud-based Hosting Service / Platform.
Location	Runs locally on a developer's machine.	Hosted on remote cloud servers.
Primary Function	Tracks local file history, branches, and revisions.	Stores Git repos online for team sharing & management.
Interface	Command Line Interface (CLI) / Terminal.	Web Graphical User Interface (GUI).
Extra Features	Core VCS capabilities only.	Issues, Pull Requests, Actions (CI/CD), Pages hosting.

Question 9: What is the staging area in Git and why do we need it?

Answer:

Definition:

The Staging Area is an intermediate preview area between the local Working Directory and the Git Repository.

Why We Need It:

- Granular Commits: Allows you to select and commit specific modified files while leaving other modified files uncommitted.
- Review Changes: Serves as a checkpoint to inspect exactly what changes will enter the repository using `git diff --staged` before saving them permanently.
- Commit Crafting: Enables grouping related changes into logical, single-topic commits rather than dumping all local changes at once.

Question 10: What is the difference between `git fetch` and `git pull`?

Answer:

Key Distinction: `git pull` is essentially a combination of `git fetch` followed immediately by `git merge`.

Aspect	git fetch	git pull
Operation	Downloads remote commits without modifying local working files.	Downloads remote commits and merges them directly into local working branch.
Safety	Safe: Lets you review changes before integrating.	Less Safe: Can create immediate merge conflicts if local files clash.
Local Impact	Working directory remains untouched.	Working directory updates immediately with remote code.
Formula	<code>git fetch</code>	<code>git fetch + git merge</code>